

2.1 Cart-Pole Swing-Up with Limited Actuation

2.1(a)

We want the constraint $s_{k+1} = f_d(s_k, u_k)$ to have the form $s_{k+1} = A_k s_k + B_k u_k + c_k$ to derive the convex approximation of the OCP at iteration i , around $(s_k^{(i)}, u_k^{(i)})$.

The linearization of $f_d(s_k, u_k)$ becomes:

$$f_d(s_k^{(i)}, u_k^{(i)}) = f(s_k^{(i)}, u_k^{(i)}) + \frac{\partial f_d}{\partial s}(s_k^{(i)}, u_k^{(i)})(s_k - s_k^{(i)}) + \frac{\partial f_d}{\partial u}(s_k^{(i)}, u_k^{(i)})(u_k - u_k^{(i)})$$

After expanding terms, we come up with

$$A_k = \frac{\partial f_d}{\partial s}(s_k^{(i)}, u_k^{(i)}), \quad B_k = \frac{\partial f_d}{\partial u}(s_k^{(i)}, u_k^{(i)}),$$

$$c_k = f_d(s_k^{(i)}, u_k^{(i)}) - A_k s_k^{(i)} - B_k u_k^{(i)}.$$

The convex subproblem is

$$\begin{aligned} \min_{s, u} \quad & \sum_{k=0}^{N-1} ((s_k - s_{\text{goal}})^T Q (s_k - s_{\text{goal}}) + u_k^T R u_k) + (s_N - s_{\text{goal}})^T P (s_N - s_{\text{goal}}) \\ \text{s.t.} \quad & s_0 = \bar{s}_0, \\ & s_{k+1} = A_k s_k + B_k u_k + c_k, \\ & -u_{\max} \leq u_k \leq u_{\max}, \\ & \|s_k - s_k^{(i)}\|_{\infty} \leq \rho, \quad \|u_k - u_k^{(i)}\|_{\infty} \leq \rho. \end{aligned}$$

2.1(b)

```

1 A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
2 c = f(s, u) - A @ s - B @ u

```

2.1(d)

First use SCP to compute a feasible swing-up trajectory. Once the pendulum is near $(0, \pi, 0, 0)$, switch to a local LQR controller designed around $(0, \pi, 0, 0)$ for stabilization.

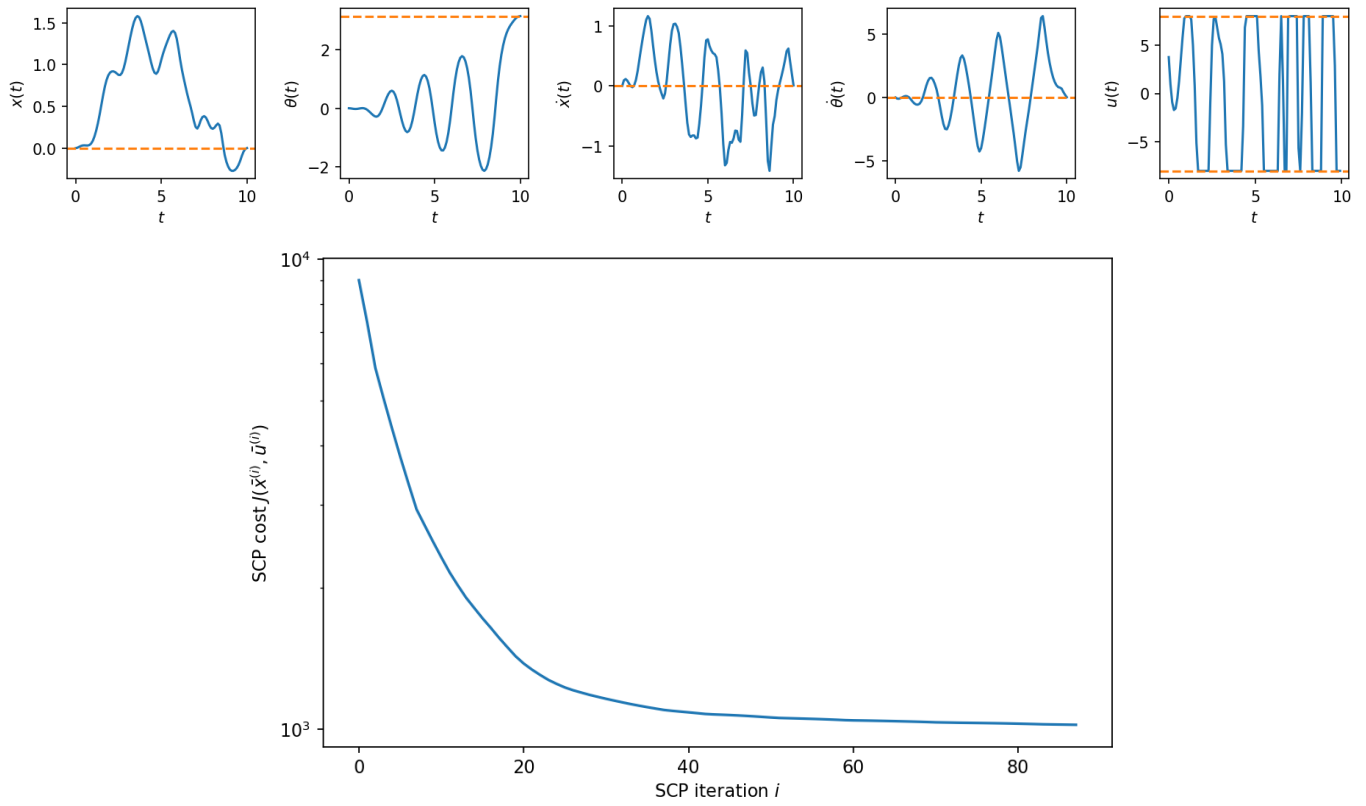


Figure 1: SCP swing-up trajectory with bounded input and trust-region updates.

Code

Code in Appendix: `cartpole_swingup_constrained.py`.

2.2 Shortest Path Through a Grid

2.2(a)

Following DP, the next table, based on Fig. 2, with each node cost is computed.

node	$J(\text{node})$
B	0
m	10
n	11
j	18
k	16
l	20
f	25
g	23
h	21
i	27
c	29
d	29
e	31
a	35
b	36
A	40

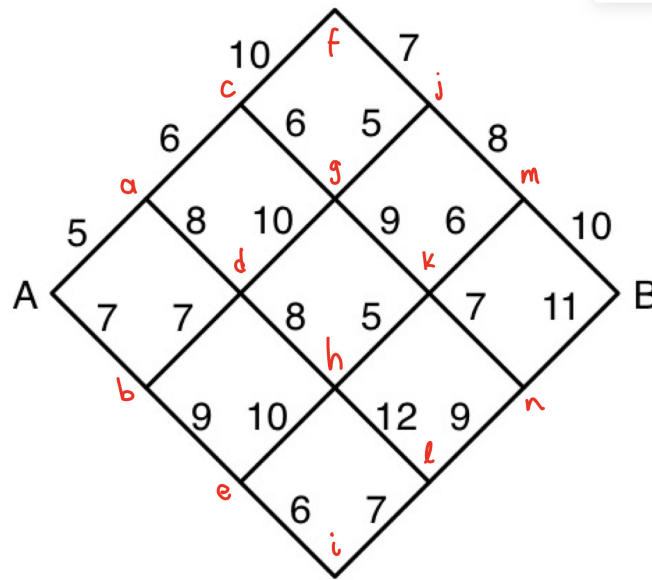


Figure 2: Named shortest-path grid used for the DP value table

The shortest path has cost 40. And the optimal path is:

$$A \rightarrow a \rightarrow c \rightarrow g \rightarrow j \rightarrow m \rightarrow B.$$

2.2(b)

For an n -segment grid, each route consists of n up moves and n down moves, so exhaustive search evaluates

$\boxed{\binom{2n}{n}}$ routes.

Dynamic programming evaluates each nonterminal node. The number of DP evaluations is $\boxed{n(n+2)}$.

For $n = 3$, these give $\binom{6}{3} = 20$ and $3(5) = 15$.

2.3 Cart-Pole Balance

2.3(a)

Start with the Euler approximation

$$s_{k+1} \approx s_k + \Delta t f(s_k, u_k)$$

Now linearize around $\bar{s} = (0, \pi, 0, 0)$, $\bar{u} = 0$, and use $\tilde{s}_k = s_k - \bar{s}$. This gives

$$\tilde{s}_{k+1} \approx \left(I + \Delta t \frac{\partial f}{\partial s}(\bar{s}, \bar{u}) \right) \tilde{s}_k + \Delta t \frac{\partial f}{\partial u}(\bar{s}, \bar{u}) u_k$$

So the discrete-time linearized system has

$$A = I + \Delta t \frac{\partial f}{\partial s}(\bar{s}, \bar{u}), \quad B = \Delta t \frac{\partial f}{\partial u}(\bar{s}, \bar{u})$$

Plugging in the Jacobians from the problem statement gives

$$\tilde{s}_{k+1} \approx A \tilde{s}_k + B u_k$$

where

$$A = I + \Delta t \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{m_p g}{m_c} & 0 & 0 \\ 0 & \frac{(m_c + m_p)g}{m_c \ell} & 0 & 0 \end{bmatrix}, \quad B = \Delta t \begin{bmatrix} 0 \\ 0 \\ \frac{1}{m_c} \\ \frac{1}{m_c \ell} \end{bmatrix}$$

2.3(b)

Using Riccati recursion until $\|P_k - P_{k-1}\|_{\max} < 10^{-4}$ gives

$$K: \begin{bmatrix} 0.73 & -231.85 & 4.22 & -68.24 \end{bmatrix}$$

2.3(c)

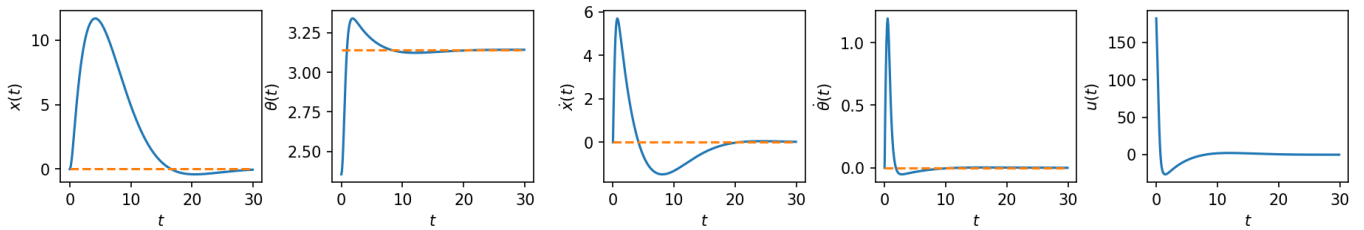


Figure 3: Infinite-horizon LQR controller for cat-pole balance

2.3(d)

i. We can reuse A and B because the dynamics Jacobians along the reference do not depend on x , and the desired trajectory keeps $\theta = \pi$, $\dot{\theta} = 0$. Then the same upright linearization is valid along the reference.

ii.

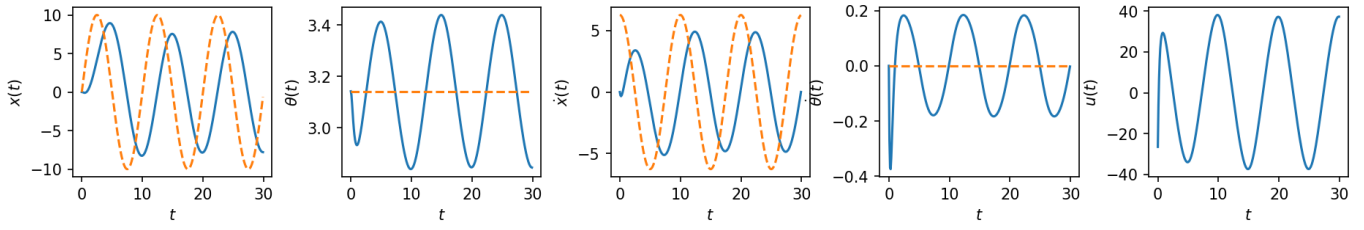


Figure 4: Tracking new reference trajectory $s(t)$ with LQR feedback.

iii. The pendulum must tilt to generate the horizontal cart acceleration from the reference, so keeping $\theta(t) \equiv \pi$ while moving the cart sinusoidally is not a dynamically consistent upright trajectory without feedforward compensation.

Code

Code in Appendix: `cartpole_balance.py`.

2.4 Cart-Pole Swing-Up

2.4(a)

```
1 A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
```

2.4(b)

First write the actual trajectory as the nominal trajectory plus a deviation

$$\tilde{s}_k = s_k - \bar{s}_k, \quad \tilde{u}_k = u_k - \bar{u}_k$$

or equivalently

$$s_k = \bar{s}_k + \tilde{s}_k, \quad u_k = \bar{u}_k + \tilde{u}_k$$

For the state part of the stage cost, this gives

$$\begin{aligned} \frac{1}{2}(s_k - s_{\text{goal}})^T Q (s_k - s_{\text{goal}}) &= \frac{1}{2}(\tilde{s}_k + \bar{s}_k - s_{\text{goal}})^T Q (\tilde{s}_k + \bar{s}_k - s_{\text{goal}}) \\ &= \frac{1}{2}\tilde{s}_k^T Q \tilde{s}_k + (\bar{s}_k - s_{\text{goal}})^T Q \tilde{s}_k + \text{constant} \end{aligned}$$

The input cost expands the same way

$$\begin{aligned} \frac{1}{2}u_k^T R u_k &= \frac{1}{2}(\bar{u}_k + \tilde{u}_k)^T R (\bar{u}_k + \tilde{u}_k) \\ &= \frac{1}{2}\tilde{u}_k^T R \tilde{u}_k + \bar{u}_k^T R \tilde{u}_k + \text{constant} \end{aligned}$$

The terminal state cost is also the same pattern

$$\frac{1}{2}(s_N - s_{\text{goal}})^T Q_N (s_N - s_{\text{goal}}) = \frac{1}{2}\tilde{s}_N^T Q_N \tilde{s}_N + (\bar{s}_N - s_{\text{goal}})^T Q_N \tilde{s}_N + \text{constant}$$

After dropping constants, the linear terms are

$$q_N = Q_N(\bar{s}_N - s_{\text{goal}}), \quad q_k = Q(\bar{s}_k - s_{\text{goal}}), \quad r_k = R\bar{u}_k$$

2.4(d)

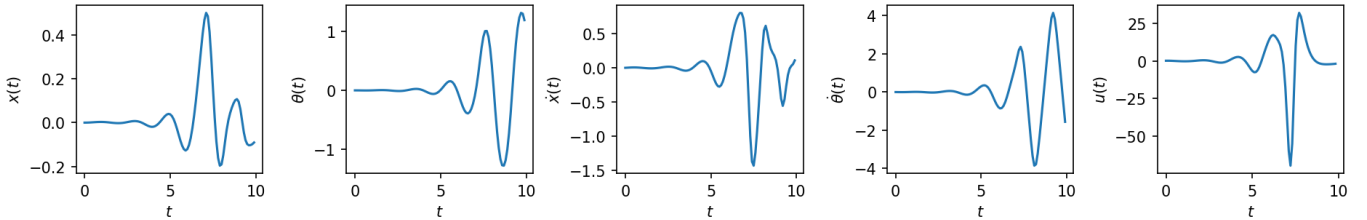


Figure 5: Open-loop application of the iLQR control sequence.

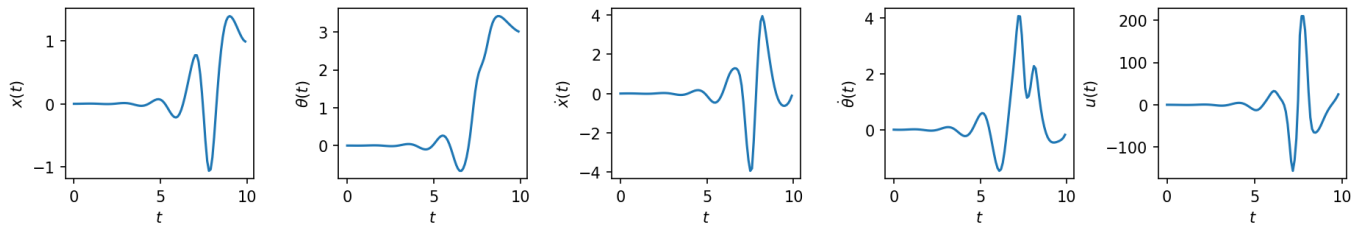


Figure 6: Closed-loop iLQR policy.

Code in Appendix: `cartpole_swingup.py`.

2.5 Machine Maintenance

This is a finite-horizon stochastic DP problem. The state is the condition of the machine at the start of a week: running R or broken B . The action is chosen after observing this state, so the solution is a closed-loop policy rather than one fixed open-loop action sequence.

Let $V_R(h)$ and $V_B(h)$ be the optimal expected profit with h weeks remaining when the machine starts the week running or broken. Since this problem asks for profit, the Bellman recursion uses a maximum:

$$V_h(x) = \max_{u \in U(x)} \mathbb{E} [\text{one-week profit} + V_{h-1}(x')]$$

For a running machine, the two choices are preventive maintenance M and no maintenance N . The one-step action values are

$$\begin{aligned} Q_R^M(h) &= 0.6(80 + V_R(h-1)) + 0.4(-20 + V_B(h-1)) \\ Q_R^N(h) &= 0.3(100 + V_R(h-1)) + 0.7V_B(h-1) \end{aligned}$$

Here $80 = 100 - 20$ is the profit if maintenance is done and the machine runs, and -20 is the maintenance cost if it still breaks.

For a broken machine, the two choices are repair C and replacement P . The corresponding action values are

$$\begin{aligned} Q_B^C(h) &= 0.6(60 + V_R(h-1)) + 0.4(-40 + V_B(h-1)) \\ Q_B^P(h) &= -50 + V_R(h-1) \end{aligned}$$

The repair value uses $60 = 100 - 40$, while replacement gives a new machine that is guaranteed to run for the current week, so its immediate net profit is $100 - 150 = -50$.

The DP update is therefore

$$\begin{aligned} V_R(h) &= \max \{40 + 0.6V_R(h-1) + 0.4V_B(h-1), 30 + 0.3V_R(h-1) + 0.7V_B(h-1)\} \\ V_B(h) &= \max \{20 + 0.6V_R(h-1) + 0.4V_B(h-1), -50 + V_R(h-1)\} \end{aligned}$$

with terminal values $V_R(0) = V_B(0) = 0$.

Working backward gives the action values

h	$Q_R^M(h)$	$Q_R^N(h)$	$Q_B^C(h)$	$Q_B^P(h)$
1	40	30	20	-50
2	72	56	52	-10
3	104	88	84	22

so the value functions are

h	0	1	2	3
$V_R(h)$	0	40	72	104
$V_B(h)$	0	20	52	84

At each remaining horizon, preventive maintenance is better when the machine is running, and repair is better when the machine is broken.

The first week is handled separately because the machine is new and guaranteed to run through that week. After receiving the first week's profit, there are three weeks left with the machine running, so the total expected profit is

$$\boxed{100 + V_R(3) = 204}$$

Code Appendix

Homework part	Code file
2.1	cartpole_swingup_constrained.py
2.3	cartpole_balance.py
2.4	cartpole_swingup.py

cartpole_swingup_constrained.py

```
1  """
2  Starter code for the problem "Cart-pole swing-up with limited actuation".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7  from functools import partial
8  from pathlib import Path
9
10 from animations import animate_cartpole
11
12 import cvxpy as cvx
13
14 import jax
15 import jax.numpy as jnp
16
17 import matplotlib.pyplot as plt
18
19 import numpy as np
20
21 from tqdm import tqdm
22
23 FIG_DIR = Path(__file__).resolve().parents[1] / "latex" / "figures"
24 FIG_DIR.mkdir(parents=True, exist_ok=True)
25
26 @partial(jax.jit, static_argnums=(0,))
27 @partial(jax.vmap, in_axes=(None, 0, 0))
28 def affinize(f, s, u):
29     """Affinize the function 'f(s,u)' around '(s,u)'.
```

```

52 # INSTRUCTIONS: Use JAX to affinize 'f' around '(s, u)' in two lines.
53 A, B = jax.jacobian(f, args=(0, 1))(s, u)
54 c = f(s, u) - A @ s - B @ u
55 # END PART (b) #####
56 return A, B, c
57
58
59 def solve_swingup_scp(f, s0, s_goal, N, P, Q, R, u_max,  $\rho$ , eps, max_iters):
60     """Solve the cart-pole swing-up problem via SCP.
61
62     Arguments
63     -----
64     f: callable
65         A function describing the discrete-time dynamics, such that
66         's[k+1] = f(s[k], u[k])'.
67     s0: numpy.ndarray
68         The initial state (1-D).
69     s_goal: numpy.ndarray
70         The goal state (1-D).
71     N: int
72         The time horizon of the LQR cost function.
73     P: numpy.ndarray
74         The terminal state cost matrix (2-D).
75     Q: numpy.ndarray
76         The state stage cost matrix (2-D).
77     R: numpy.ndarray
78         The control stage cost matrix (2-D).
79     u_max: float
80         The bound defining the control set '[-u_max, u_max]'.
81      $\rho$ : float
82         Trust region radius.
83     eps: float
84         Termination threshold for SCP.
85     max_iters: int
86         Maximum number of SCP iterations.
87
88     Returns
89     -----
90     s: numpy.ndarray
91         A 2-D array where 's[k]' is the open-loop state at time step 'k',
92         for 'k=0, 1, ..., N-1'.
93     u: numpy.ndarray
94         A 2-D array where 'u[k]' is the open-loop state at time step 'k',
95         for 'k=0, 1, ..., N-1'.
96     J: numpy.ndarray
97         A 1-D array where 'J[i]' is the SCP sub-problem cost after the i-th
98         iteration, for 'i=0, 1, ..., (iteration when convergence occurred)'.
99     """
100     n = Q.shape[0] # state dimension
101     m = R.shape[0] # control dimension
102
103     # Initialize dynamically feasible nominal trajectories
104     u = np.zeros((N, m))
105     s = np.zeros((N + 1, n))
106     s[0] = s0
107     for k in range(N):
108         s[k + 1] = fd(s[k], u[k])
109
110     # Do SCP until convergence or maximum number of iterations is reached
111     converged = False
112     J = np.zeros(max_iters + 1)
113     J[0] = np.inf
114     for i in (prog_bar := tqdm(range(max_iters))):

```

```

115     s, u, J[i + 1] = scp_iteration(f, s0, s_goal, s, u, N, P, Q, R, u_max, ρ)
116     dJ = np.abs(J[i + 1] - J[i])
117     prog_bar.set_postfix({"objective_change": "{:.5f}".format(dJ)})
118     if dJ < eps:
119         converged = True
120         print("SCP converged after {} iterations.".format(i))
121         break
122     if not converged:
123         raise RuntimeError("SCP did not converge!")
124     J = J[1 : i + 1]
125     return s, u, J
126
127
128 def scp_iteration(f, s0, s_goal, s_prev, u_prev, N, P, Q, R, u_max, ρ):
129     """Solve a single SCP sub-problem for the cart-pole swing-up problem.
130
131     Arguments
132     -----
133     f: callable
134     A function describing the discrete-time dynamics, such that
135     's[k+1] = f(s[k], u[k])'.
136     s0: numpy.ndarray
137     The initial state (1-D).
138     s_goal: numpy.ndarray
139     The goal state (1-D).
140     s_prev: numpy.ndarray
141     The state trajectory around which the problem is convexified (2-D).
142     u_prev: numpy.ndarray
143     The control trajectory around which the problem is convexified (2-D).
144     N: int
145     The time horizon of the LQR cost function.
146     P: numpy.ndarray
147     The terminal state cost matrix (2-D).
148     Q: numpy.ndarray
149     The state stage cost matrix (2-D).
150     R: numpy.ndarray
151     The control stage cost matrix (2-D).
152     u_max: float
153     The bound defining the control set '[-u_max, u_max]'.
154     ρ: float
155     Trust region radius.
156
157     Returns
158     -----
159     s: numpy.ndarray
160     A 2-D array where 's[k]' is the open-loop state at time step 'k',
161     for 'k=0, 1, ..., N-1'.
162     u: numpy.ndarray
163     A 2-D array where 'u[k]' is the open-loop state at time step 'k',
164     for 'k=0, 1, ..., N-1'.
165     J: float
166     The SCP sub-problem cost.
167     """
168     A, B, c = affinize(f, s_prev[:-1], u_prev)
169     A, B, c = np.array(A), np.array(B), np.array(c)
170     n = Q.shape[0]
171     m = R.shape[0]
172     s_cvx = cvx.Variable((N + 1, n))
173     u_cvx = cvx.Variable((N, m))
174
175     # PART (c) #####
176     # INSTRUCTIONS: Construct the convex SCP sub-problem.
177     objective = cvx.quad_form(s_cvx[N] - s_goal, P)

```

```

178 constraints = [s_cvx[0] == s0]
179 for k in range(N):
180     objective += cvx.quad_form(s_cvx[k] - s_goal, Q)
181     objective += cvx.quad_form(u_cvx[k], R)
182     constraints += [
183         s_cvx[k + 1] == A[k] @ s_cvx[k] + B[k] @ u_cvx[k] + c[k],
184         cvx.abs(u_cvx[k, 0]) <= u_max,
185         cvx.norm_inf(s_cvx[k] - s_prev[k]) <= ρ,
186         cvx.norm_inf(u_cvx[k] - u_prev[k]) <= ρ,
187     ]
188 constraints.append(cvx.norm_inf(s_cvx[N] - s_prev[N]) <= ρ)
189 # END PART (c) #####
190
191 prob = cvx.Problem(cvx.Minimize(objective), constraints)
192 prob.solve()
193 if prob.status != "optimal":
194     raise RuntimeError("SCP_solve_failed. Problem status: " + prob.status)
195 s = s_cvx.value
196 u = u_cvx.value
197 J = prob.objective.value
198 return s, u, J
199
200
201 def cartpole(s, u):
202     """Compute the cart-pole state derivative."""
203     mp = 1.0 # pendulum mass
204     mc = 4.0 # cart mass
205     L = 1.0 # pendulum length
206     g = 9.81 # gravitational acceleration
207
208     x, θ, dx, dθ = s
209     sinθ, cosθ = jnp.sin(θ), jnp.cos(θ)
210     h = mc + mp * (sinθ**2)
211     ds = jnp.array(
212         [
213             dx,
214             dθ,
215             (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
216             -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
217             / (h * L),
218         ]
219     )
220     return ds
221
222
223 def discretize(f, dt):
224     """Discretize continuous-time dynamics 'f' via Runge-Kutta integration."""
225
226     def integrator(s, u, dt=dt):
227         k1 = dt * f(s, u)
228         k2 = dt * f(s + k1 / 2, u)
229         k3 = dt * f(s + k2 / 2, u)
230         k4 = dt * f(s + k3, u)
231         return s + (k1 + 2 * k2 + 2 * k3 + k4) / 6
232
233     return integrator
234
235
236 # Define constants
237 n = 4 # state dimension
238 m = 1 # control dimension
239 s_goal = np.array([0, np.pi, 0, 0]) # desired upright pendulum state
240 s0 = np.array([0, 0, 0, 0]) # initial downright pendulum state

```

```

241 dt = 0.1 # discrete time resolution
242 T = 10.0 # total simulation time
243 P = 1e3 * np.eye(n) # terminal state cost matrix
244 Q = np.diag([1e-2, 1.0, 1e-3, 1e-3]) # state cost matrix
245 R = 1e-3 * np.eye(m) # control cost matrix
246 ρ = 1.0 # trust region parameter
247 u_max = 8.0 # control effort bound
248 eps = 5e-1 # convergence tolerance
249 max_iters = 100 # maximum number of SCP iterations
250 animate = True # flag for animation
251
252 # Initialize the discrete-time dynamics
253 fd = jax.jit(discretize(cartpole, dt))
254
255 # Solve the swing-up problem with SCP
256 t = np.arange(0.0, T + dt, dt)
257 N = t.size - 1
258 s, u, J = solve_swingup_scp(fd, s0, s_goal, N, P, Q, R, u_max, ρ, eps, max_iters)
259
260 # Simulate open-loop control
261 for k in range(N):
262     s[k + 1] = fd(s[k], u[k])
263
264 # Plot state and control trajectories
265 fig, ax = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
266 plt.subplots_adjust(wspace=0.45)
267 labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
268 labels_u = (r"$u(t)$",)
269 for i in range(n):
270     ax[i].plot(t, s[:, i])
271     ax[i].axhline(s_goal[i], linestyle="--", color="tab:orange")
272     ax[i].set_xlabel(r"$t$")
273     ax[i].set_ylabel(labels_s[i])
274 for i in range(m):
275     ax[n + i].plot(t[:-1], u[:, i])
276     ax[n + i].axhline(u_max, linestyle="--", color="tab:orange")
277     ax[n + i].axhline(-u_max, linestyle="--", color="tab:orange")
278     ax[n + i].set_xlabel(r"$t$")
279     ax[n + i].set_ylabel(labels_u[i])
280 plt.savefig(FIG_DIR / "cartpole_swingup_constrained.png", bbox_inches="tight")
281
282 # Plot cost history over SCP iterations
283 fig, ax = plt.subplots(1, 1, dpi=150, figsize=(8, 5))
284 ax.semilogy(J)
285 ax.set_xlabel(r"SCP_iteration_$i$")
286 ax.set_ylabel(r"SCP_cost_$J(\bar{x}^{(i)}, \bar{u}^{(i)})$")
287 plt.savefig(FIG_DIR / "cartpole_swingup_constrained_cost.png", bbox_inches="tight")
288 # plt.show()
289
290 # Animate the solution
291 if animate:
292     fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
293     ani.save(FIG_DIR / "cartpole_swingup_constrained.mp4", writer="ffmpeg")
294     # plt.show()

```

cartpole_balance.py

```

1 """
2 Solution code for the problem "Cart-pole balance".
3
4 Autonomous Systems Lab (ASL), Stanford University

```

```

5  """
6
7  import numpy as np
8  from scipy.integrate import odeint
9  import matplotlib.pyplot as plt
10 from pathlib import Path
11
12 from animations import animate_cartpole
13
14 # Constants
15 n = 4 # state dimension
16 m = 1 # control dimension
17 mp = 2.0 # pendulum mass
18 mc = 10.0 # cart mass
19 L = 1.0 # pendulum length
20 g = 9.81 # gravitational acceleration
21 dt = 0.1 # discretization time step
22 animate = True # whether or not to animate results
23
24 FIG_DIR = Path(__file__).resolve().parents[1] / "latex" / "figures"
25 FIG_DIR.mkdir(parents=True, exist_ok=True)
26
27 def cartpole(s: np.ndarray, u: np.ndarray) -> np.ndarray:
28     """Compute the cart-pole state derivative
29
30     Args:
31         s (np.ndarray): The cartpole state: [x, theta, x_dot, theta_dot], shape (n,)
32         u (np.ndarray): The cartpole control: [F_x], shape (m,)
33
34     Returns:
35         np.ndarray: The state derivative, shape (n,)
36     """
37     x, theta, dx, dtheta = s
38     sintheta, costheta = np.sin(theta), np.cos(theta)
39     h = mc + mp * (sintheta**2)
40     ds = np.array(
41         [
42             dx,
43             dtheta,
44             (mp * sintheta * (L * (dtheta**2) + g * costheta) + u[0]) / h,
45             -((mc + mp) * g * sintheta + mp * L * (dtheta**2) * sintheta * costheta + u[0] * costheta)
46             / (h * L),
47         ]
48     )
49     return ds
50
51
52 def reference(t: float) -> np.ndarray:
53     """Compute the reference state (s_bar) at time t
54
55     Args:
56         t (float): Evaluation time
57
58     Returns:
59         np.ndarray: Reference state, shape (n,)
60     """
61     a = 10.0 # Amplitude
62     T = 10.0 # Period
63
64     # PART (d) #####
65     # INSTRUCTIONS: Compute the reference state for a given time
66     omega = 2 * np.pi / T
67     return np.array([a * np.sin(omega * t), np.pi, a * omega * np.cos(omega * t), 0.0])

```

```

68 # END PART (d) #####
69
70
71 def ricatti_recursion(
72     A: np.ndarray, B: np.ndarray, Q: np.ndarray, R: np.ndarray
73 ) -> np.ndarray:
74     """Compute the gain matrix K through Ricatti recursion
75
76     Args:
77         A(np.ndarray): Dynamics matrix, shape (n, n)
78         B(np.ndarray): Controls matrix, shape (n, m)
79         Q(np.ndarray): State cost matrix, shape (n, n)
80         R(np.ndarray): Control cost matrix, shape (m, m)
81
82     Returns:
83         np.ndarray: Gain matrix K, shape (m, n)
84     """
85     eps = 1e-4 # Riccati recursion convergence tolerance
86     max_iters = 1000 # Riccati recursion maximum number of iterations
87     P_prev = np.zeros((n, n)) # initialization
88     converged = False
89     for i in range(max_iters):
90         # PART (b) #####
91         # INSTRUCTIONS: Apply the Ricatti equation until convergence
92         K = -np.linalg.solve(R + B.T @ P_prev @ B, B.T @ P_prev @ A)
93         P = Q + A.T @ P_prev @ (A + B @ K)
94         if np.max(np.abs(P - P_prev)) < eps:
95             converged = True
96             break
97         P_prev = P
98     # END PART (b) #####
99     if not converged:
100         raise RuntimeError("Ricatti recursion did not converge!")
101     print("K:", K)
102     return K
103
104
105 def simulate(
106     t: np.ndarray, s_ref: np.ndarray, u_ref: np.ndarray, s0: np.ndarray, K: np.ndarray
107 ) -> tuple[np.ndarray, np.ndarray]:
108     """Simulate the cartpole
109
110     Args:
111         t(np.ndarray): Evaluation times, shape (num_timesteps,)
112         s_ref(np.ndarray): Reference state s_bar, evaluated at each time t. Shape (
113             num_timesteps, n)
114         u_ref(np.ndarray): Reference control u_bar, shape (m,)
115         s0(np.ndarray): Initial state, shape (n,)
116         K(np.ndarray): Feedback gain matrix (Ricatti recursion result), shape (m, n)
117
118     Returns:
119         tuple[np.ndarray, np.ndarray]: Tuple of:
120         np.ndarray: The state history, shape (num_timesteps, n)
121         np.ndarray: The control history, shape (num_timesteps, m)
122     """
123
124     def cartpole_wrapper(s, t, u):
125         """Helper function to get cartpole() into a form preferred by odeint, which
126             expects t as the second arg"""
127         return cartpole(s, u)
128
129     # PART (c) #####
130     # INSTRUCTIONS: Complete the function to simulate the cartpole system

```



```

129 # Hint: use the cartpole wrapper above with odeint
130 s = np.zeros((t.size, n))
131 u = np.zeros((t.size, m))
132 s[0] = s0
133 for k in range(t.size - 1):
134     u[k] = u_ref + K @ (s[k] - s_ref[k])
135     s[k + 1] = odeint(cartpole_wrapper, s[k], t[k : k + 2], args=(u[k],))[1]
136 u[-1] = u[-2]
137 # END PART (c) #####
138 return s, u
139
140
141 def compute_lti_matrices() -> tuple[np.ndarray, np.ndarray]:
142     """Compute the linearized dynamics matrices A and B of the LTI system
143
144     Returns:
145     tuple[np.ndarray, np.ndarray]: Tuple of:
146     np.ndarray: The A (dynamics) matrix, shape (n, n)
147     np.ndarray: The B (controls) matrix, shape (n, m)
148     """
149     # PART (a) #####
150     # INSTRUCTIONS: Construct the A and B matrices
151     a = mp * g / mc
152     b = (mc + mp) * g / (mc * L)
153     F = np.array(
154         [
155             [0.0, 0.0, 1.0, 0.0],
156             [0.0, 0.0, 0.0, 1.0],
157             [0.0, a, 0.0, 0.0],
158             [0.0, b, 0.0, 0.0],
159         ]
160     )
161     A = np.eye(n) + dt * F
162     B = dt * np.array([[0.0], [0.0], [1.0 / mc], [1.0 / (mc * L)]])
163     # END PART (a) #####
164     return A, B
165
166
167 def plot_state_and_control_history(
168     s: np.ndarray, u: np.ndarray, t: np.ndarray, s_ref: np.ndarray, name: str
169 ) -> None:
170     """Helper function for cartpole visualization
171
172     Args:
173     s(np.ndarray): State history, shape (num_timesteps, n)
174     u(np.ndarray): Control history, shape (num_timesteps, m)
175     t(np.ndarray): Times, shape (num_timesteps,)
176     s_ref(np.ndarray): Reference state s_bar, evaluated at each time t. Shape (
177         num_timesteps, n)
178     name(str): Filename prefix for saving figures
179     """
180     fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
181     plt.subplots_adjust(wspace=0.35)
182     labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
183     labels_u = (r"$u(t)$",)
184     for i in range(n):
185         axes[i].plot(t, s[:, i])
186         axes[i].plot(t, s_ref[:, i], "--")
187         axes[i].set_xlabel(r"$t$")
188         axes[i].set_ylabel(labels_s[i])
189     for i in range(m):
190         axes[n + i].plot(t, u[:, i])
191         axes[n + i].set_xlabel(r"$t$")

```

```

191     axes[n + i].set_ylabel(labels_u[i])
192 plt.savefig(FIG_DIR / f"{name}.png", bbox_inches="tight")
193 # plt.show()
194
195 if animate:
196     fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
197     ani.save(FIG_DIR / f"{name}.mp4", writer="ffmpeg")
198     # plt.show()
199
200
201 def main():
202     # Part A
203     A, B = compute_lti_matrices()
204
205     # Part B
206     Q = np.eye(n) # state cost matrix
207     R = np.eye(m) # control cost matrix
208     K = ricatti_recursion(A, B, Q, R)
209
210     # Part C
211     t = np.arange(0.0, 30.0, 1 / 10)
212     s_ref = np.array([0.0, np.pi, 0.0, 0.0]) * np.ones((t.size, 1))
213     u_ref = np.array([0.0])
214     s0 = np.array([0.0, 3 * np.pi / 4, 0.0, 0.0])
215     s, u = simulate(t, s_ref, u_ref, s0, K)
216     plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance")
217
218     # Part D
219     # Note: t, u_ref unchanged from part c
220     s_ref = np.array([reference(ti) for ti in t])
221     s0 = np.array([0.0, np.pi, 0.0, 0.0])
222     s, u = simulate(t, s_ref, u_ref, s0, K)
223     plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance_tv")
224
225
226 if __name__ == "__main__":
227     main()

```

cartpole_swingup.py

```

1  """
2  Starter code for the problem "Cart-pole swing-up".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7  import time
8  from pathlib import Path
9
10 from animations import animate_cartpole
11
12 import jax
13 import jax.numpy as jnp
14
15 import matplotlib.pyplot as plt
16
17 import numpy as np
18
19 from scipy.integrate import odeint
20
21 FIG_DIR = Path(__file__).resolve().parents[1] / "latex" / "figures"

```

```

22 FIG_DIR.mkdir(parents=True, exist_ok=True)
23
24 def linearize(f, s, u):
25     """Linearize the function 'f(s,u)' around '(s,u)' .
26
27     Arguments
28     -----
29     f: callable
30         A nonlinear function with call signature 'f(s,u)' .
31     s: numpy.ndarray
32         The state (1-D) .
33     u: numpy.ndarray
34         The control input (1-D) .
35
36     Returns
37     -----
38     A: numpy.ndarray
39         The Jacobian of 'f' at '(s,u)', with respect to 's' .
40     B: numpy.ndarray
41         The Jacobian of 'f' at '(s,u)', with respect to 'u' .
42     """
43     # WRITE YOUR CODE BELOW #####
44     # INSTRUCTIONS: Use JAX to compute 'A' and 'B' in one line.
45     A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
46     #####
47     return A, B
48
49
50 def ilqr(f, s0, s_goal, N, Q, R, QN, eps=1e-3, max_iters=1000):
51     """Compute the iLQR set-point tracking solution.
52
53     Arguments
54     -----
55     f: callable
56         A function describing the discrete-time dynamics, such that
57         's[k+1] = f(s[k], u[k])' .
58     s0: numpy.ndarray
59         The initial state (1-D) .
60     s_goal: numpy.ndarray
61         The goal state (1-D) .
62     N: int
63         The time horizon of the LQR cost function .
64     Q: numpy.ndarray
65         The state cost matrix (2-D) .
66     R: numpy.ndarray
67         The control cost matrix (2-D) .
68     QN: numpy.ndarray
69         The terminal state cost matrix (2-D) .
70     eps: float, optional
71         Termination threshold for iLQR .
72     max_iters: int, optional
73         Maximum number of iLQR iterations .
74
75     Returns
76     -----
77     s_bar: numpy.ndarray
78         A 2-D array where 's_bar[k]' is the nominal state at time step 'k',
79         for 'k=0, 1, ..., N-1'
80     u_bar: numpy.ndarray
81         A 2-D array where 'u_bar[k]' is the nominal control at time step 'k',
82         for 'k=0, 1, ..., N-1'
83     Y: numpy.ndarray
84         A 3-D array where 'Y[k]' is the matrix gain term of the iLQR control

```

```

85 law_at_time_step_k', for k=0,1,...,N-1'
86 y:=numpy.ndarray
87 A2-Darraywhere y[k] 'is the offset term of the iLQR control law
88 law_at_time_step_k', for k=0,1,...,N-1'
89 """
90 if max_iters <= 1:
91     raise ValueError("Argument 'max_iters' must be at least 1.")
92 n = Q.shape[0] # state dimension
93 m = R.shape[0] # control dimension
94
95 # Initialize gains 'Y' and offsets 'y' for the policy
96 Y = np.zeros((N, m, n))
97 y = np.zeros((N, m))
98
99 # Initialize the nominal trajectory '(s_bar, u_bar)', and the
100 # deviations '(ds, du)'
101 u_bar = np.zeros((N, m))
102 s_bar = np.zeros((N + 1, n))
103 s_bar[0] = s0
104 for k in range(N):
105     s_bar[k + 1] = f(s_bar[k], u_bar[k])
106 ds = np.zeros((N + 1, n))
107 du = np.zeros((N, m))
108
109 # iLQR loop
110 converged = False
111 for _ in range(max_iters):
112     # Linearize the dynamics at each step 'k' of '(s_bar, u_bar)'
113     A, B = jax.vmap(linearize, in_axes=(None, 0, 0))(f, s_bar[:-1], u_bar)
114     A, B = np.array(A), np.array(B)
115
116     # PART (c) #####
117     # INSTRUCTIONS: Update 'Y', 'y', 'ds', 'du', 's_bar', and 'u_bar'.
118     q = [Q @ (s_bar[k] - s_goal) for k in range(N)]
119     r = [R @ u_bar[k] for k in range(N)]
120     P = QN.copy()
121     p = QN @ (s_bar[N] - s_goal)
122
123     for k in reversed(range(N)):
124         G = R + B[k].T @ P @ B[k]
125         Y[k] = -np.linalg.solve(G, B[k].T @ P @ A[k])
126         y[k] = -np.linalg.solve(G, r[k] + B[k].T @ p)
127         p = q[k] + A[k].T @ p + A[k].T @ P @ B[k] @ y[k]
128         P = Q + A[k].T @ P @ (A[k] + B[k] @ Y[k])
129
130     ds.fill(0.0)
131     du.fill(0.0)
132     for k in range(N):
133         du[k] = Y[k] @ ds[k] + y[k]
134         ds[k + 1] = A[k] @ ds[k] + B[k] @ du[k]
135
136     def rollout_cost(s_candidate, u_candidate):
137         J = 0.5 * (s_candidate[N] - s_goal) @ QN @ (s_candidate[N] - s_goal)
138         for j in range(N):
139             J += 0.5 * (s_candidate[j] - s_goal) @ Q @ (s_candidate[j] - s_goal)
140             J += 0.5 * u_candidate[j] @ R @ u_candidate[j]
141         return J
142
143     J_prev = rollout_cost(s_bar, u_bar)
144     accepted = False
145     for alpha in (1.0, 0.5, 0.25, 0.1, 0.05, 0.01):
146         u_next = u_bar + alpha * du
147         s_next = np.zeros_like(s_bar)

```

```

148         s_next[0] = s0
149         for k in range(N):
150             s_next[k + 1] = f(s_next[k], u_next[k])
151             if np.all(np.isfinite(s_next)) and rollout_cost(s_next, u_next) < J_prev:
152                 accepted = True
153                 du *= alpha
154                 u_bar = u_next
155                 s_bar = s_next
156                 break
157             if not accepted:
158                 du.fill(0.0)
159             #####
160
161             if np.max(np.abs(du)) < eps:
162                 converged = True
163                 break
164             if not converged:
165                 raise RuntimeError("iLQR did not converge!")
166         return s_bar, u_bar, Y, y
167
168
169 def cartpole(s, u):
170     """Compute the cart-pole state derivative."""
171     mp = 2.0 # pendulum mass
172     mc = 10.0 # cart mass
173     L = 1.0 # pendulum length
174     g = 9.81 # gravitational acceleration
175
176     x,  $\theta$ , dx, d $\theta$  = s
177     sin $\theta$ , cos $\theta$  = jnp.sin( $\theta$ ), jnp.cos( $\theta$ )
178     h = mc + mp * (sin $\theta$ **2)
179     ds = jnp.array(
180         [
181             dx,
182             d $\theta$ ,
183             (mp * sin $\theta$  * (L * (d $\theta$ **2) + g * cos $\theta$ ) + u[0]) / h,
184             -((mc + mp) * g * sin $\theta$  + mp * L * (d $\theta$ **2) * sin $\theta$  * cos $\theta$  + u[0] * cos $\theta$ )
185             / (h * L),
186         ]
187     )
188     return ds
189
190
191 # Define constants
192 n = 4 # state dimension
193 m = 1 # control dimension
194 Q = np.diag(np.array([10.0, 10.0, 2.0, 2.0])) # state cost matrix
195 R = 1e-2 * np.eye(m) # control cost matrix
196 QN = 1e2 * np.eye(n) # terminal state cost matrix
197 s0 = np.array([0.0, 0.0, 0.0, 0.0]) # initial state
198 s_goal = np.array([0.0, np.pi, 0.0, 0.0]) # goal state
199 T = 10.0 # simulation time
200 dt = 0.1 # sampling time
201 animate = True # flag for animation
202 closed_loop = True # flag for closed-loop control
203
204 # Initialize continuous-time and discretized dynamics
205 f = jax.jit(cartpole)
206 fd = jax.jit(lambda s, u, dt=dt: s + dt * f(s, u))
207
208 # Compute the iLQR solution with the discretized dynamics
209 print("Computing iLQR solution...", end="", flush=True)
210 start = time.time()

```

```

211 t = np.arange(0.0, T, dt)
212 N = t.size - 1
213 s_bar, u_bar, Y, y = ilqr(fd, s0, s_goal, N, Q, R, QN)
214 print("done!_({:.2f}_s)".format(time.time() - start), flush=True)
215
216 # Simulate on the true continuous-time system
217 print("Simulating_...", end="", flush=True)
218 start = time.time()
219 s = np.zeros((N + 1, n))
220 u = np.zeros((N, m))
221 s[0] = s0
222 for k in range(N):
223     # PART (d) #####
224     # INSTRUCTIONS: Compute either the closed-loop or open-loop value of
225     # 'u[k]', depending on the Boolean flag 'closed_loop'.
226     if closed_loop:
227         u[k] = u_bar[k] + Y[k] @ (s[k] - s_bar[k]) + y[k]
228     else: # do open-loop control
229         u[k] = u_bar[k]
230     #####
231     s[k + 1] = odeint(lambda s, t: f(s, u[k]), s[k], t[k : k + 2])[1]
232 print("done!_({:.2f}_s)".format(time.time() - start), flush=True)
233
234 # Plot
235 fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
236 plt.subplots_adjust(wspace=0.45)
237 labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
238 labels_u = (r"$u(t)$",)
239 for i in range(n):
240     axes[i].plot(t, s[:, i])
241     axes[i].set_xlabel(r"$t$")
242     axes[i].set_ylabel(labels_s[i])
243 for i in range(m):
244     axes[n + i].plot(t[:-1], u[:, i])
245     axes[n + i].set_xlabel(r"$t$")
246     axes[n + i].set_ylabel(labels_u[i])
247 if closed_loop:
248     plt.savefig(FIG_DIR / "cartpole_swingup_cl.png", bbox_inches="tight")
249 else:
250     plt.savefig(FIG_DIR / "cartpole_swingup_ol.png", bbox_inches="tight")
251 # plt.show()
252
253 if animate:
254     fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
255     ani.save(FIG_DIR / "cartpole_swingup.mp4", writer="ffmpeg")
256     # plt.show()

```